

6809/6800 and FLEX: Using MicroC (Part 1)

 flexonsbd.blogspot.com/2018/05/microc.html

Friday, May 4, 2018

Using MicroC (Part 1)

MicroC source

FLEX9 used to have a famous integer-type C compiler called MicroC, created by Masataka Ota. I remember it being sold commercially as well.

A few years ago, I got a book called "C Language and Peripheral Control" (by Yasushi Niitsu and Kozo Ikegami, Kogakusha) which uses MicroC for control, and I really wanted to try it out.

The source code and its assembler can be downloaded from [Mr. Takeoka's website](#). However, this is for BSD, so it cannot be built directly on Linux, which I use regularly.

Building MicroC source code on Linux

- (1) Compile mc.c with gcc > mc (a cross-compiler that runs on Linux)
- (2) Compile mc2.c with mc -> c.out (assembly listing created by the cross compiler)
- (3) Assemble c.txt in as09 > mc2.o (Motorola S format object)
- (4) Convert to binary format using mot2bin -> rename to mc09, and remove the leading \$00~\$FF as it is not needed (binary executable file)

Change the two filenames that conflict with the declaration in stdio.h.

 getline() -> getline2()

This change means that during the make process, `gcc mc.c -O mc` will create the `mc` file, but it won't be saved.

Next, if you change all instances of `printf(` in `mc.c` to `fprintf(obuf,`, the output will be saved to the file `c.out`.

Finally, transfer the created mc09 and C.TXT files, along with the libraries c.txt, stdio.txt, stdio2.txt, alloc.txt, fileio.txt, fileio2.txt, scanf.txt, string.txt, and the utility uf.c, to FLEX.

(This is based on notes from 2014, and there may be differences in recent versions of Ubuntu 16, such as the index() method no longer conflicting...)

Use mc09

(1) Notes on Source Creation

By default, `stdio.txt` and `alloc.txt` must be included. (File names must be in uppercase.)

Also, since the assembler ASMB treats TAB codes as something else, convert them to Space.

Furthermore, convert `$0D` and `$0A` at the end of lines to just `$0D`.

(2)

Build with `mc09 -O1.C.OUT file.c` to create `C.OUT` on drive 1. (The name `C.OUT` cannot be changed.)

(3) Assemble

Place `C.TXT` on drive 1 beforehand.

ASMB `C.TXT +YLS` will read `C.OUT` during the assembly of `C.TXT` and assemble it to create `C.bin`.

(4) Execution

The resulting `C.bin` is loaded from `$100` and executed from `$100`, so it can be executed as is.